

PROYECTO INTERMODULAR 2

SPRINT 1 - FASE DE DESARROLLO E IMPLEMENTACIÓN TÉCNICA

Miriam Mendieta Racedo.-

2DAMS

10/10/2026



SPRINT 1 - FASE DE DESARROLLO E IMPLEMENTACIÓN TÉCNICA

- Proyecto: ZeertyDigital.
- Autor@: Miriam Mendieta Racedo.-
- Fecha: 10 de Octubre de 2026
- IES Serra Perenxisa - Torrent

INDICE

INTRODUCCIÓN	1
RESTRUCTURACIÓN DE HU POR PRIORIDAD (METODOLOGÍA MOSCOW)	2
RIESGOS Y MITIGACIONES DEL PROYECTO	4
DIAGRAMAS UML (LENGUAJE UNIFICADO DE MODELADO)	5
1.Diagrama de Casos de Uso (Perspectiva del Usuario)	5
2.Diagrama de Clases	8
3.Diagrama de Actividades (Flujo del Proceso)	10
4.Diagrama de Objetos	11
MODELO ENTIDAD-RELACIÓN (ER)	12
EVOLUCIÓN DEL TRABAJO Y ESTADO DE HISTORIAS DE USUARIO (SPRINT 1)	13
A.Historias de Usuario Completadas	13
B.Dificultades / Impedimentos Técnicos Solventados	13
C.Tareas No Finalizadas Traspasadas al Sprint 2	13
BIBLIOGRAFÍA	14

INTRODUCCIÓN

Con las bases conceptuales y los requisitos operativos establecidos, iniciamos la Fase de Desarrollo e Implementación Técnica, núcleo práctico de este proyecto de Gestión de Activos Digitales: ***ZeertyDigital***.

Para responder a las necesidades de autónomos y pequeños comercios con bajo perfil técnico, se ha diseñado una arquitectura basada en Java Spring Boot, Thymeleaf y Mysql empaquetada íntegramente en Docker.

Esta estrategia permite aislar por completo el entorno de ejecución, garantizando que la solución pueda desplegarse en la máquina del cliente con un único comando o instalador, sin requerir la configuración de servicios externos ni la instalación previa de Java. A lo largo de esta sección se detalla el desarrollo de la aplicación, el modelado de la persistencia embebida y la creación de la imagen del contenedor para su posterior distribución, así como también un apartado con la evolución del proceso del trabajo realizado



Reestructuración de HU por Prioridad (metodología MoSCoW)

He realizado una reestructuración de las HU aplicando el método MoSCoW. Esto me ayudó a tener una visión realista de la implementación de los módulos y en la gestión del tiempo, ya que dicha metodología junto con Scrum es especialmente útil cuando los plazos son ajustados y es necesario para saber rápidamente sobre lo qué es crítico.

Prioridad	HU incluidas	Estado
MUST (PMV)	HU-01.x, HU-02.x, HU-03.x, HU-04.1, HU-04.3, HU-06.x, HU-07.1	Se desarrollan ahora
SHOULD	HU-04.2, HU-07.2	Se desarrollan después
COULD	Módulo de Marca, Reputación, Activos Tecnológicos. IA	Opcional
WON'T (por ahora)	HU-05.1 y HU-05.2 (OCR y gestoría) Nube	Se implementan más adelante

El PMV de ZeertyDigital se ha redefinido siguiendo la metodología MoSCoW, priorizando únicamente las funcionalidades esenciales para validar el producto y garantizar que cumple su objetivo principal: permitir a cualquier autónomo o pequeño comercio inventariar, clasificar, guardar y respaldar sus activos digitales desde una interfaz centralizada, segura y fácil de usar.

Para ello, se han seleccionado únicamente los módulos que permiten:

- **Acceso seguro a la plataforma**, mediante registro, inicio de sesión y recuperación de contraseña.
- **Inventario centralizado de activos digitales**, con alta, categorización, búsqueda, edición y eliminación.
- **Gestión segura de credenciales**, con cifrado y visualización controlada.
- **Gestión documental básica**, con subida, previsualización y descarga de archivos.
- **Control del negocio**, mediante dashboard y alertas de caducidad.
- **Protección de datos**, mediante generación de copias de seguridad manuales.

PROYECTO INTERMODULAR 2



Estas funcionalidades permiten validar la utilidad real de la aplicación sin necesidad de módulos avanzados. El PMV garantiza que el usuario puede **organizar y proteger la información crítica del negocio**, evitando pérdidas de accesos, caducidades y dispersión documental. Por ello, se han excluido del PMV funcionalidades complementarias como OCR, exportación trimestral, sincronización con Drive o restauración de backups, que aportan valor, pero no son necesarias para validar el producto.

La reestructuración garantiza:

- Cumplimiento de fechas de entrega
- Un PMV sólido y demostrable
- Un vídeo final atractivo con mejoras reales
- Un desarrollo escalonado y profesional
- Una planificación alineada con MoSCoW y con la carga de trabajo realista

Funcionalidades obligatorias para que la app pueda usarse:

- HU-01.1 Registro de Usuarios: Sin registro, no existe acceso al sistema → Must Have.
- HU-01.2 Inicio y Cierre de Sesión: Es la base de la seguridad → Must Have.
- HU-01.4 Alta de Usuarios por Administrador: Necesario para gestionar roles y accesos → Must Have.
- HU-06.1 Dashboard Base: Es la pantalla principal tras iniciar sesión → Must Have.

Tareas técnicas del Sprint 1

- Diagrama ER/UML
- Docker + MySQL
- Proyecto base Spring Boot
- Seguridad (JWT/Sesiones)

Todas estas tareas son **Must Have** porque sin ellas no existe arquitectura, base de datos ni backend funcional.



Riesgos y Mitigaciones del Proyecto

La mitigación es el conjunto de acciones preventivas o correctivas que se aplican para reducir el impacto de un riesgo o evitar que ocurra durante el desarrollo del proyecto.

- * **Riesgo 1 — Sobrecarga de funcionalidades:** Intentar incluir OCR o módulos avanzados. **Mitigación:** Reestructuración MoSCoW → OCR pasa a mejora futura.
- * **Riesgo 2 — Dependencias externas (Google Drive, OCR, APIs):** Integraciones que pueden fallar o requerir más tiempo. **Mitigación:** Mantener PMV sin dependencias externas.
- * **Riesgo 3 — Complejidad del cifrado y seguridad:** Implementación incorrecta de cifrado puede comprometer datos. **Mitigación:** Sprint 3 dedicado exclusivamente a seguridad.
- * **Riesgo 4 — Retrasos en la entregas hasta llegar al vídeo (aprox. 10/11):** Si el PMV no está estable o las entregas se retrasan por una mala gestión del tiempo, el vídeo no puede grabarse. **Mitigación:** PMV completo en Sprint 3 → Sprint 4 solo para vídeo.
- * **Riesgo 5 — Falta de datos reales para pruebas:** Sin activos reales, el sistema no se valida correctamente. **Mitigación:** Checklist de auditoría inicial integrada en el documento.
- * **Riesgo 6 — Problemas de integración entre módulos:** Fallos al unir usuarios, activos, credenciales y documentos. **Mitigación:** Sprint 4 dedicado a pruebas de integración.

Diagramas UML (Lenguaje Unificado de Modelado)

El uso de diagramas UML proporciona un lenguaje visual estandarizado que permite representar de forma clara la estructura, el comportamiento y las interacciones de un sistema. Incorporar esta modelación en este momento del desarrollo aporta orden, claridad y previsión. Esto se traduce en un menor margen de error, la reducción de costes de mantenimiento y una base documental sólida que actúa como puente entre la arquitectura conceptual y la implementación en código, garantizando un software más robusto, coherente y fácil de evolucionar.

1. Diagrama de Casos de Uso (Perspectiva del Usuario)

Este diagrama ayuda a definir los límites del sistema, quiénes interactúan con la aplicación y qué acciones principales pueden realizar

Descripción de Casos de Uso Principales

- **Registrar y Clasificar Activo Digital** → Permite al usuario dar de alta un nuevo activo digital en el sistema, asignándole una categoría específica (Contenido, Software, Datos, Marketing o Activo Financiero) e introduciendo su información asociada.
 - Actor Principal: Usuario (Autónomo / Pequeño Comercio o empleado autorizado).
 - Precondiciones: El usuario debe estar autenticado e identificado en el sistema.
 - Flujo Principal:
 1. El usuario accede a la sección "Nuevo Activo" del panel centralizado.
 2. El sistema muestra el formulario de registro con la selección de categorías.
 3. El usuario selecciona el tipo de activo e introduce los datos requeridos (nombre, descripción, ubicación/URL, valor estimado y fecha de registro).
 4. El sistema valida los datos obligatorios.
 5. Datos incompletos o erróneos: se muestra un mensaje de error destacando los campos requeridos y mantiene el formulario activo.
 6. El sistema persiste el activo en la base de datos y actualiza el panel de control.

7. Postcondiciones: El activo digital queda inventariado y visible en el panel del usuario.

- **Gestionar Credenciales y Accesos Cifrados** → Permite almacenar y consultar contraseñas, claves API y tokens de acceso vinculados a los activos digitales, aplicando cifrado de extremo a extremo previo a su almacenamiento.
 - Actor Principal: Usuario (Autónomo / Pequeño Comercio, personal autorizado).
 - Precondiciones: El usuario tiene una sesión activa y ha ingresado la clave maestra de descifrado local.
 - Flujo Principal:
 1. El usuario selecciona la opción "Añadir Credencial" dentro de un activo registrado.
 2. El usuario introduce la URL de servicio, usuario y contraseña/clave.
 3. Se cifra los datos sensibles utilizando el algoritmo de encriptación.
 4. El sistema almacena la cadena cifrada en la base de datos.
 5. Para consultar la credencial, el usuario solicita "Mostrar Contraseña" y el sistema la descripta en memoria de forma temporal.
 6. Clave maestra incorrecta: El sistema deniega el descifrado y solicita reintroducir la clave.
 7. Postcondiciones: La credencial queda almacenada en formato inaccesible en la base de datos, garantizando cero exposición de texto plano.

- **Monitoreo y Gestión de Alertas de Vencimiento** → Evalúa periódicamente los activos digitales con fechas límite (certificados SSL, dominios, licencias, contratos) y envía notificaciones preventivas para evitar la caducidad inadvertida.
 - Actor Principal: Sistema (Proceso automático).
 - Actor Secundario: Usuario (Receptor de la alerta).
 - Precondiciones: Existen activos registrados con fechas de expiración válidas en la base de datos.
 - Flujo Principal:
 1. El servicio en segundo plano del sistema se ejecuta de manera programada (diariamente).
 2. El sistema consulta los activos cuyas fechas de vencimiento estén dentro del umbral de alerta (30, 15 y 7 días previos).

3. El sistema genera una alerta en el panel de control y envía una notificación por correo al usuario.
4. El usuario recibe la alerta y puede marcar el activo como "Renovado" actualizando la nueva fecha de vencimiento.
5. Postcondiciones: El usuario es informado con antelación, previniendo interrupciones de servicio o sanciones.

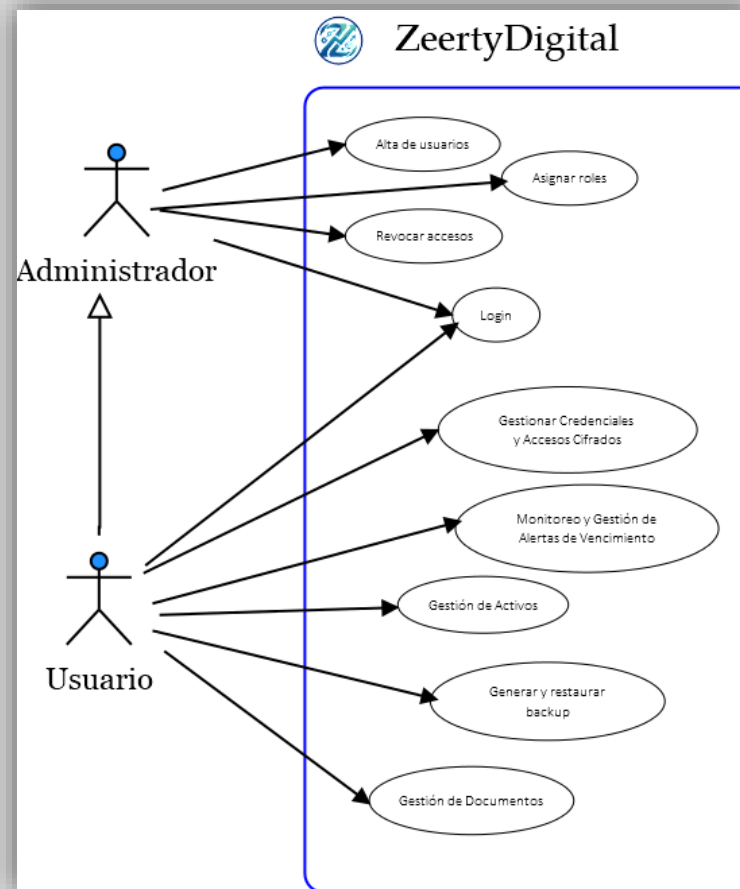


Diagrama de Casos de Uso

2. Diagrama de Clases

Representa la arquitectura estática de la app, se modela el dominio del problema traduciendo los requerimientos del negocio en entidades de software, definiendo sus atributos, métodos principales, modificadores de acceso y las relaciones de asociación, herencia y composición entre ellas.

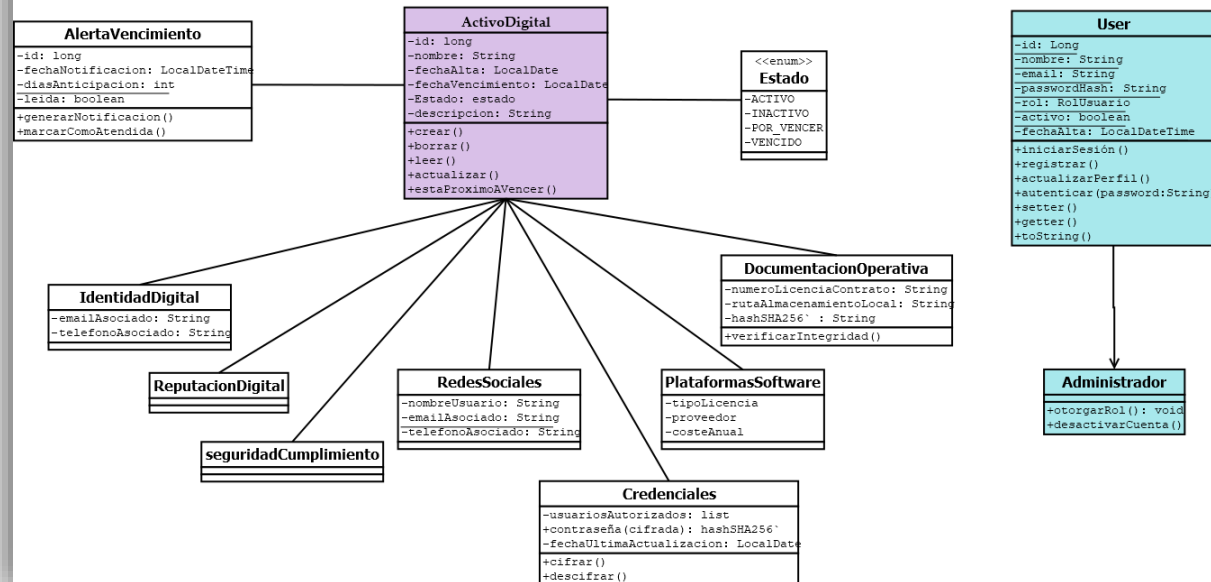
Clases de Entidad

- **UsuarioApp:** Representa al usuario registrado o administrador del sistema.
- **ActivoDigital:** Entidad central del inventario (recurso registrado).
- **Categoria:** Clasificación del activo digital (Software, Redes Sociales, Dominios, etc.).
- **Documento:** Ficheros adjuntos.
- **Etiqueta (Tag):** Esencial para los motores de búsqueda internos. Mantiene una relación de *Muchos a Muchos* con los activos (un activo puede tener muchas etiquetas, y una etiqueta puede pertenecer a muchos activos)
- **Backup:** Registro de las copias de seguridad generadas e importadas.

Enumerados / Enums

- **EstadoActivo:** Estado del activo (ACTIVO, POR_VENCER, VENCIDO, INACTIVO).
- **Categorias:** MARKETING, LEGAL, FINANZAS, TECNOLOGIA, RGPD_DATOS, CONFIDENCIAL, ACCESO_PUBLICO

DIAGRAMA DE CLASES



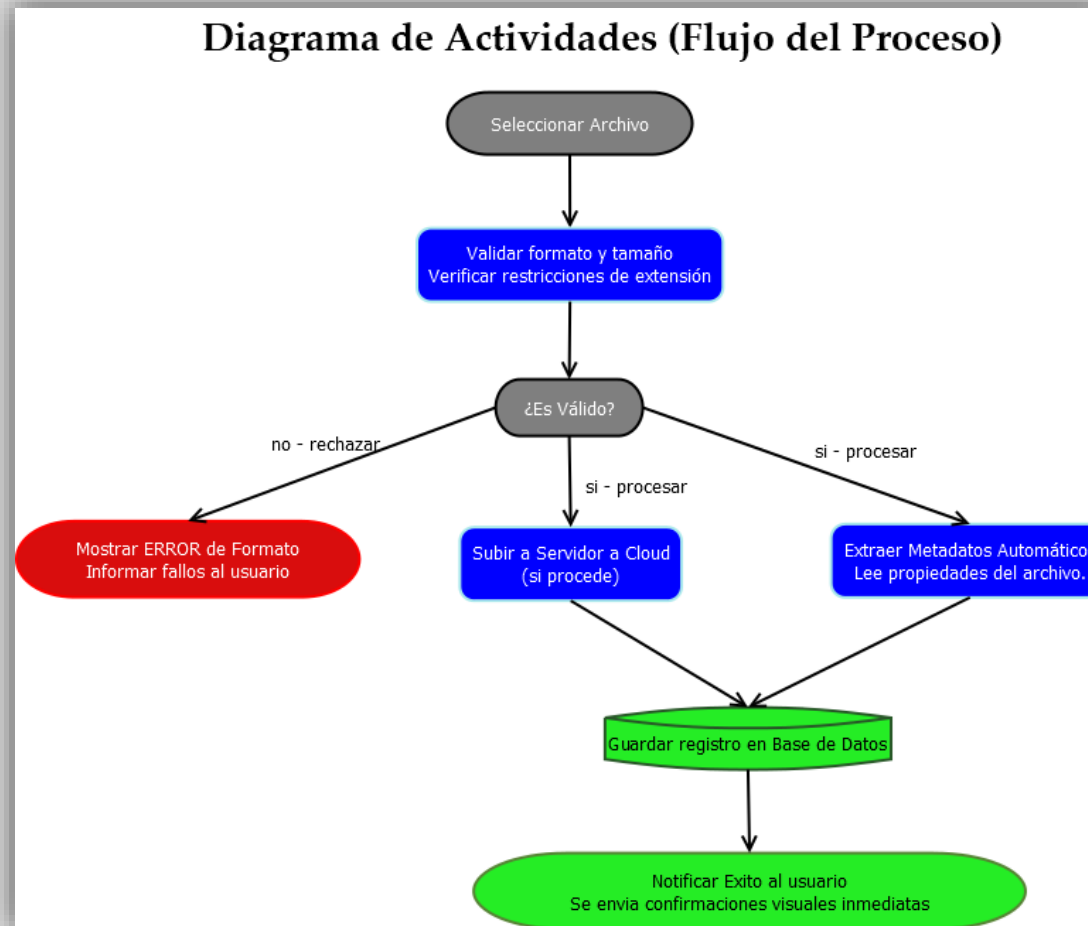
Relaciones y Cardinalidad entre Entidades

- Usuario — (1 : N) — ActivoDigital(Composición) → Un usuario posee de cero a múltiples (0..*) activos digitales. Cada activo digital pertenece de manera unívoca (1) a un único usuario.
- ActivoDigital — (1 : N) — AlertaVencimiento (Asociación Simple) → Un activo con fecha de expiración puede programar múltiples alertas temporales (0..*) a medida que se aproximan sus fechas límites (ej. a 30 días, 15 días y 3 días de la expiración).



3. Diagrama de Actividades (Flujo del Proceso)

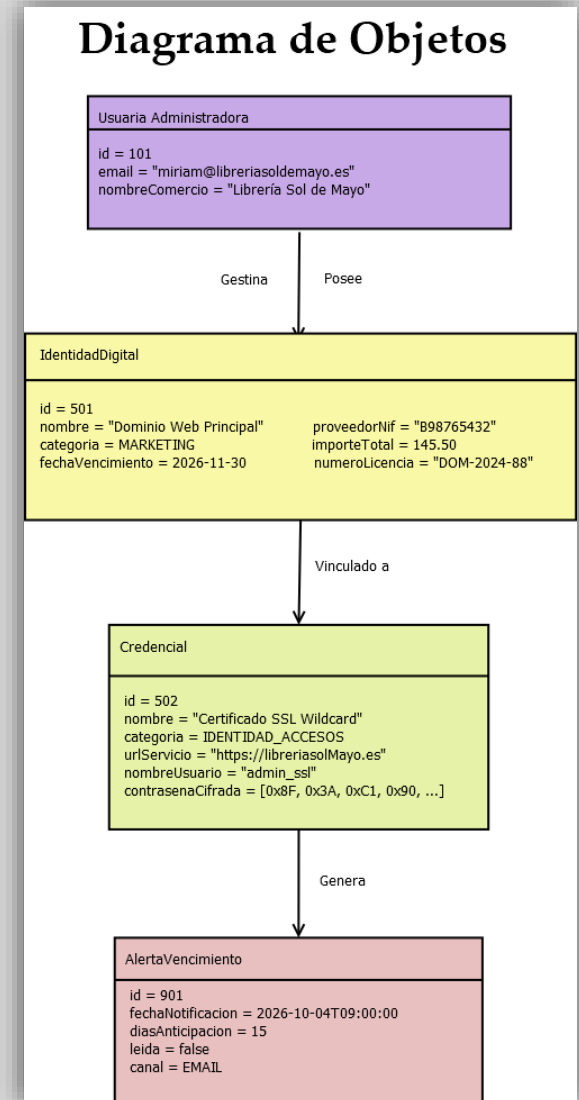
Este modelo describe el proceso dinámico que ocurre desde el momento en que un usuario arrastra un archivo a la interfaz hasta que queda guardado y catalogado con éxito.



4. Diagrama de Objetos

A diferencia del Diagrama de Clases, que muestra la estructura abstracta del software, el Diagrama de Objetos captura un estado concreto del sistema en un instante de tiempo específico. Muestra instancias reales de las clases (objetos) con valores asignados a sus atributos y los enlaces que existen entre ellas durante una operación real.

El siguiente diagrama de objetos se ilustra como ejemplo el estado de la memoria del sistema durante la gestión diaria del comercio "Librería Sol de Mayo" (perteneciente a la usuaria Miriam). En este instante preciso, el sistema gestiona un dominio web, un certificado SSL cifrado próximo a vencer (que ha generado una alerta).



Modelo Entidad-Relación (ER)

Es una representación visual de cómo se organizan y conectan los datos dentro de un sistema. Su importancia radica en que permite estructurar la información de forma clara, evitar duplicidades (redundancia) y asegurar la integridad de la base de datos antes de su implementación física.

El Modelo ER define la estructura lógica de los datos del sistema, permitiendo:

- Organizar los activos digitales y recursos del negocio de forma coherente.
- Garantizar la integridad referencial y las relaciones entre entidades.
- Facilitar el diseño de operaciones CRUD y el rendimiento de consultas eficientes.
- Preparar la estructura base para la creación física del esquema en MySQL.

El sistema deberá gestionar:

- Usuarios y roles
- Activos digitales
- Categorías
- Estados
- Documentos asociados
- Credenciales cifradas
- Copias de seguridad (Backups)

Evolución del Trabajo y Estado de Historias de Usuario (Sprint 1)

A. Historias de Usuario Completadas

B. Dificultades / Impedimentos Técnicos Solventados

Dificultades: gestión del tiempo, no saber que datos a guardar en la Base de datos

C. Tareas No Finalizadas Traspasadas al Sprint 2

BIBLIOGRAFÍA

- Apuntes del Profesor.
- Taller de SCRUM-PI2S. Septiembre 2026. Fco. Javier Tárrega Andreu.
- <https://www.itdo.com/blog/moscow-que-es-y-como-priorizar-en-el-desarrollo-de-tu-aplicacion/>
- <https://www.youtube.com/watch?v=QMkRme6bEgE>
- <https://www.youtube.com/watch?v=lwNaN1vOKs4>
- <https://bonzzay.com/blog/stack-tecnologico-que-es-como-funciona/>
- Curso UML: <https://www.youtube.com/playlist?list=PLU8oAIHdN5BmmxXT0C2HO0bLRHZFWKbhH>
- Editor de diagramas: <http://dia-installer.de/download/index.html>
-

